```
// ===== File: vite.config.ts =====
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

// https://vite.dev/config/
export default defineConfig({
  plugins: [react()],
  server: {
    port: 5173,
    // 代理配置 - 开发环境将 /api 请求转发到后端服务（server.js 监听 8080，路由自带 /api 前缀）
    proxy: {
      '/api': {
        target: 'http://localhost:8080',
        changeOrigin: true,
      },
    },
    cors: true,
  },
  // qiankun 子应用配置
  base: '/',
  build: {
    target: 'esnext',
    outDir: 'dist',
    assetsDir: 'assets',
    rollupOptions: {
      output: {
        manualChunks(id: string) {
          if (id.includes('node_modules')) {
            if (id.includes('three')) return 'three';
            if (id.includes('echarts')) return 'echarts';
            return 'vendor';
          }
        },
      },
    },
  },
})

// ===== File: vitest.config.ts =====
 /// <reference types="vitest" />
 import { defineConfig } from 'vitest/config'
 import react from '@vitejs/plugin-react'

 export default defineConfig({
   plugins: [react()],
   test: {
     environment: 'jsdom',
     globals: true,
     setupFiles: ['./src/test/setup.ts'],
     include: ['src/**/*.test.{ts,tsx}'],
```

```
    css: false,
  },
 })

// ===== File: eslint.config.js =====
import js from '@eslint/js'
import globals from 'globals'
import reactHooks from 'eslint-plugin-react-hooks'
import reactRefresh from 'eslint-plugin-react-refresh'
import tseslint from 'typescript-eslint'
import { defineConfig, globalIgnores } from 'eslint/config'

export default defineConfig([
  globalIgnores(['dist']),
  {
    files: ['**/*.{ts,tsx}'],
    extends: [
      js.configs.recommended,
      tseslint.configs.recommended,
      reactHooks.configs.flat.recommended,
      reactRefresh.configs.vite,
    ],
    languageOptions: {
      ecmaVersion: 2020,
      globals: globals.browser,
    },
  },
])

// ===== File: index.html =====
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>react-ai-chat</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>

// ===== File: src\App.tsx =====
/**
 * 青宸智汇 Chat - 主应用组件
 * 左侧边栏导航：功能区 + AI 角色区
 */
```

```
import React, { useState, useEffect } from 'react';
import { Layout, Menu, Typography, Tooltip, Space, Button, Spin } from 'antd';
import {
  PlusOutlined,
  BookOutlined,
  ApiOutlined,
  ThunderboltOutlined,
  SearchOutlined,
  BulbOutlined,
  ToolOutlined,
  CustomerServiceOutlined,
  DatabaseOutlined,
  AppstoreOutlined,
  BarChartOutlined,
  BellOutlined,
} from '@ant-design/icons';
import { useAIStore } from './store/aiStore';
import { useAuthStore } from './store/authStore';
import { useNotificationStore } from './store/notificationStore';
import { useI18n } from './i18n';
import LoginPage from './pages/LoginPage';
import NotificationCenter from './components/NotificationCenter';
import type { AppPage, AIRole } from './types';
import ScoutAI from './pages/ScoutAI';
import SageAI from './pages/SageAI';
import MakerAI from './pages/MakerAI';
import ButlerAI from './pages/ButlerAI';
import NewConversation from './pages/NewConversation';
import SkillLibrary from './pages/SkillLibrary';
import MCPConfig from './pages/MCPConfig';
import Automation from './pages/Automation';
import KnowledgeVault from './pages/KnowledgeVault';
import AppCenter from './pages/AppCenter';
import UsageStats from './pages/UsageStats';
import IndustryReport from './pages/IndustryReport';
import AIPolicyPage from './pages/AIPolicy';
import IndustryDataPage from './pages/IndustryData';
import MemoryPanel from './components/MemoryPanel';
import UserProfileMenu from './components/UserProfileMenu';
import SettingsDrawer from './components/SettingsDrawer';

const { Sider, Content } = Layout;
const { Text } = Typography;

// 快捷功能区配置（文案走 i18n）
const makeToolMenuItems = (t: (k: string) => string) => [
  {
    key: 'new-conversation',
    label: t('menu.newConversation'),
    icon: <PlusOutlined />,
```

```
    description: t('menu.newConversationDesc'),
  },
  {
    key: 'skill-library',
    label: t('menu.skillLibrary'),
    icon: <BookOutlined />,
    description: t('menu.skillLibraryDesc'),
  },
  {
    key: 'mcp-config',
    label: t('menu.mcp'),
    icon: <ApiOutlined />,
    description: t('menu.mcpDesc'),
  },
  {
    key: 'automation',
    label: t('menu.automation'),
    icon: <ThunderboltOutlined />,
    description: t('menu.automationDesc'),
  },
  {
    key: 'knowledge-vault',
    label: t('menu.knowledgeVault'),
    icon: <DatabaseOutlined />,
    description: t('menu.knowledgeVaultDesc'),
  },
  {
    key: 'app-center',
    label: t('menu.appCenter'),
    icon: <AppstoreOutlined />,
    description: t('menu.appCenterDesc'),
  },
  {
    key: 'usage-stats',
    label: t('menu.usageStats'),
    icon: <BarChartOutlined />,
    description: t('menu.usageStatsDesc'),
  },
  {
    key: 'message-center',
    label: t('menu.messageCenter'),
    icon: <BellOutlined />,
    description: t('menu.messageCenterDesc'),
  },
];

// AI 角色区配置（名称走 i18n）
const makeAiRoleItems = (t: (k: string) => string): Array<{ key: string; label: string; icon: React.ReactNode; color: st
  { key: 'ai-scout', label: t('roles.scout'), icon: <SearchOutlined />, color: '#1677ff', secondary: '#36cfc9', role: 's
  { key: 'ai-sage', label: t('roles.sage'), icon: <BulbOutlined />, color: '#faad14', secondary: '#ffc53d', role: 'sage'
```

```
  { key: 'ai-maker', label: t('roles.maker'), icon: <ToolOutlined />, color: '#52c41a', secondary: '#95de64', role: 'mak
  { key: 'ai-butler', label: t('roles.butler'), icon: <CustomerServiceOutlined />, color: '#eb2f96', secondary: '#ff85c0
];

const pageComponents: Record<AppPage, React.FC> = {
  'ai-scout': ScoutAI,
  'ai-sage': SageAI,
  'ai-maker': MakerAI,
  'ai-butler': ButlerAI,
  'new-conversation': NewConversation,
  'skill-library': SkillLibrary,
  'mcp-config': MCPConfig,
  'automation': Automation,
  'knowledge-vault': KnowledgeVault,
  'app-center': AppCenter,
  'usage-stats': UsageStats,
  'industry-report': IndustryReport,
  'ai-policy': AIPolicyPage,
  'industry-data': IndustryDataPage,
};

const App: React.FC = () => {
  const { currentPage, setCurrentPage, currentRole } = useAIStore();
  const { isAuthenticated, authLoading, restoreSession } = useAuthStore();
  const notificationOpen = useNotificationStore((s) => s.open);
  const { t } = useI18n();
  const [memoryOpen, setMemoryOpen] = useState(false);

  const toolMenuItems = makeToolMenuItems(t);
  const aiRoleItems = makeAiRoleItems(t);

  // 启动时恢复登录会话
  useEffect(() => {
    restoreSession();
  }, [restoreSession]);

  // 会话校验中：全局加载态
  if (authLoading) {
    return (
      <div
        style={{
          height: '100vh',
          display: 'flex',
          alignItems: 'center',
          justifyContent: 'center',
          background: '#0b0f19',
        }}
      >
        <Spin size="large" />
      </div>
```

```
  );
}

// 根级守卫：未登录渲染登录页
if (!isAuthenticated) {
  return <LoginPage />;
}

const handleMenuClick = (key: string) => {
  // 消息中心走 Drawer 面板，不切换页面
  if (key === 'message-center') {
    notificationOpen();
    return;
  }
  setCurrentPage(key as AppPage);
};

const PageComponent = pageComponents[currentPage];

// 获取当前角色的主题色
const currentRoleColor = aiRoleItems.find((item) => item.role === currentRole)?.color || '#1677ff';

return (
  <Layout style={{ height: '100vh' }}>
    {/* 左侧边栏 */}
    <Sider
      width={200}
      breakpoint={undefined}
      collapsible={false}
      style={{
        background: '#141414',
        borderRight: '1px solid #2a2a2a',
      }}
    >
      <div
        style={{
          display: 'flex',
          flexDirection: 'column',
          height: '100vh',
        }}
      >
        {/* Logo */}
        <div
          style={{
            padding: '20px 16px 12px',
            borderBottom: '1px solid #2a2a2a',
            flexShrink: 0,
          }}
        >
          <Space align="center" size={8}>
```

```
    <div
      className="logo-breathe"
      style={{
        width: 28,
        height: 28,
        borderRadius: 8,
        background: 'linear-gradient(135deg, #1677ff, #36cfc9)',
        display: 'flex',
        alignItems: 'center',
        justifyContent: 'center',
      }}
    >
      <Text strong style={{ color: '#fff', fontSize: 14 }}>A</Text>
    </div>
    <Text strong style={{ color: '#fff', fontSize: 16, letterSpacing: 1 }}>
      {t('app.name')}
    </Text>
  </Space>
</div>

{/* 快捷功能区 + AI 角色 */}
<div style={{ padding: '12px 8px', flex: 1, overflow: 'auto', minHeight: 0 }}>
  <Text
    type="secondary"
    style={{
      fontSize: 10,
      padding: '0 8px',
      marginBottom: 8,
      display: 'block',
      color: '#8c8c8c',
      fontWeight: 600,
      letterSpacing: 1.5,
      textTransform: 'uppercase',
    }}
  >
    {t('app.group.toolbox')}
  </Text>
  <Menu
    mode="inline"
    selectedKeys={toolMenuItems.some((i) => i.key === currentPage) ? [currentPage] : []}
    onClick={({ key }) => handleMenuClick(key)}
    style={{
      background: 'transparent',
      border: 'none',
      color: '#d9d9d9',
    }}
    items={toolMenuItems.map((item) => ({
      key: item.key,
      icon: (
        <span style={{ color: '#8c8c8c' }}>{item.icon}</span>
```

```
    ),
    label: (
      <Tooltip title={item.description} placement="right">
        <span style={{ color: '#d9d9d9' }}>{item.label}</span>
      </Tooltip>
    ),
  }))}
  theme="dark"
/>

{/* 分隔线 */}
<div style={{ margin: '16px 8px', borderTop: '1px solid #2a2a2a' }} />

<Text
  type="secondary"
  style={{
    fontSize: 10,
    padding: '0 8px',
    marginBottom: 8,
    display: 'block',
    color: '#8c8c8c',
    fontWeight: 600,
    letterSpacing: 1.5,
    textTransform: 'uppercase',
  }}
>
  {t('app.group.roles')}
</Text>
<Menu
  mode="inline"
  selectedKeys={[currentPage]}
  onClick={({ key }) => handleMenuClick(key)}
  style={{
    background: 'transparent',
    border: 'none',
  }}
  items={aiRoleItems.map((item) => ({
    key: item.key,
    icon: (
      <span style={{ color: item.color }}>{item.icon}</span>
    ),
    label: (
      <span style={{ color: '#d9d9d9' }}>{item.label}</span>
    ),
    style: {
      position: 'relative',
      marginBottom: 2,
      borderRadius: 6,
    },
  }))}
```

```
          theme="dark"
        />
      </div>

      {/* 底部：个人中心 + 记忆入口 + 版本号 */}
      <div
        style={{
          padding: '12px 16px',
          borderTop: '1px solid #2a2a2a',
          flexShrink: 0,
        }}
      >
        <Button
          block
          size="small"
          icon={<BulbOutlined />}
          onClick={() => setMemoryOpen(true)}
          style={{
            marginBottom: 8,
            background: 'transparent',
            color: '#d9d9d9',
            borderColor: '#2a2a2a',
          }}
        >
          {t('menu.memory')}
        </Button>
        <UserProfileMenu />
        <div style={{ marginTop: 8, textAlign: 'center' }}>
          <Text type="secondary" style={{ fontSize: 11, color: '#595959' }}>
            {t('app.version')}
          </Text>
        </div>
      </div>
    </div>
  </Sider>

  {/* 设置抽屉 */}
  <SettingsDrawer />

  {/* 记忆管理面板 */}
  <MemoryPanel open={memoryOpen} onClose={() => setMemoryOpen(false)} />

  {/* 消息中心面板 */}
  <NotificationCenter />

  {/* 主内容区 */}
  <Content
    key={currentPage}
    className="page-fade-in"
    style={{ background: '#f5f5f5', overflow: 'auto' }}
```

```
      >
        <PageComponent />
      </Content>
    </Layout>
  );
};

export default App;

// ===== File: src\index.css =====
/* 青宸智汇 Chat - 全局样式 */

* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, 'Helvetica Neue', Arial,
    'Noto Sans', sans-serif, 'Apple Color Emoji', 'Segoe UI Emoji';
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
}

#root {
  width: 100%;
  height: 100vh;
}

/* ========== CSS 变量系统 ========== */
:root {
  /* 角色主题色（动态覆盖） */
  --role-primary: #1677ff;
  --role-secondary: #36cfc9;

  /* 玻璃拟态 */
  --glass-bg: rgba(255, 255, 255, 0.72);
  --glass-border: rgba(255, 255, 255, 0.55);
  --glass-shadow: 0 4px 16px rgba(31, 110, 185, 0.06);

  /* 侧边栏 */
  --sider-bg: #141414;
  --sider-border: #2a2a2a;

  /* 过渡时间 */
  --transition-fast: 0.2s;
  --transition-normal: 0.3s;
  --transition-slow: 0.5s;
}
```

```
/* ========== 滚动条样式 ========== */
::-webkit-scrollbar {
  width: 6px;
  height: 6px;
}

::-webkit-scrollbar-track {
  background: transparent;
}

::-webkit-scrollbar-thumb {
  background: #d9d9d9;
  border-radius: 3px;
}

::-webkit-scrollbar-thumb:hover {
  background: #bfbfbf;
}

/* ========== Ant Design 覆盖 ========== */
.ant-layout-sider {
  transition: all 0.2s ease;
}

.ant-list-item {
  transition: background-color 0.2s ease;
}

.ant-list-item:hover {
  background-color: #f5f5f5 !important;
}

/* ========== 动画关键帧 ========== */

/* 淡入 */
@keyframes fadeIn {
  from {
    opacity: 0;
  }
  to {
    opacity: 1;
  }
}

/* 上滑淡入 */
@keyframes slideUpFade {
  from {
    opacity: 0;
    transform: translateY(12px);
```

```
  }
  to {
    opacity: 1;
    transform: translateY(0);
  }
}

/* 脉冲扩散 */
@keyframes pulseRing {
  0% {
    transform: scale(0.8);
    opacity: 0.8;
  }
  100% {
    transform: scale(2);
    opacity: 0;
  }
}

/* 呼吸动画（Logo） */
@keyframes breathe {
  0%, 100% {
    opacity: 0.85;
    filter: brightness(1);
  }
  50% {
    opacity: 1;
    filter: brightness(1.15);
  }
}

/* 渐变指示条滑入 */
@keyframes indicatorSlide {
  from {
    transform: scaleY(0);
  }
  to {
    transform: scaleY(1);
  }
}

/* ========== 页面切换动画 ========== */
.page-fade-in {
  animation: fadeIn var(--transition-fast) ease-out;
}

/* ========== 面板内容区动画 ========== */
.panel-slide-up {
  animation: slideUpFade var(--transition-normal) ease-out;
}
```

```
/* ========== 卡片悬停统一类 ========== */
.hover-card {
  transition: transform var(--transition-fast) ease,
    box-shadow var(--transition-fast) ease;
}

.hover-card:hover {
  transform: translateY(-2px);
  box-shadow: 0 8px 24px rgba(0, 0, 0, 0.12);
}

/* ========== 按钮点击反馈 ========== */
.btn-click-feedback {
  transition: transform 0.1s ease;
}

.btn-click-feedback:active {
  transform: scale(0.97);
}

/* ========== 成功脉冲动画 ========== */
.pulse-success {
  position: relative;
}

.pulse-success::after {
  content: '';
  position: absolute;
  inset: -4px;
  border-radius: inherit;
  border: 2px solid var(--role-primary);
  animation: pulseRing 0.6s ease-out forwards;
  pointer-events: none;
}

/* ========== 玻璃拟态卡片 ========== */
.glass-card {
  background: var(--glass-bg);
  backdrop-filter: blur(12px);
  -webkit-backdrop-filter: blur(12px);
  border: 1px solid var(--glass-border);
  box-shadow: var(--glass-shadow);
  border-radius: 12px;
}

/* ========== AI角色Tabs激活态指示条 ========== */
.ai-role-tabs .ant-tabs-tab-active {
  position: relative;
}
```

```
.ai-role-tabs .ant-tabs-tab-active::after {
  content: '';
  position: absolute;
  bottom: 0;
  left: 8px;
  right: 8px;
  height: 3px;
  background: linear-gradient(90deg, var(--role-primary), var(--role-secondary));
  border-radius: 2px 2px 0 0;
  animation: indicatorSlide var(--transition-fast) ease-out;
}

/* ========== 侧边栏菜单悬停指示条 ========== */
.sider-menu-item-hover {
  position: relative;
  transition: background-color var(--transition-fast) ease;
}

.sider-menu-item-hover::before {
  content: '';
  position: absolute;
  left: 0;
  top: 50%;
  transform: translateY(-50%) scaleY(0);
  width: 2px;
  height: 60%;
  background: var(--role-primary);
  border-radius: 0 2px 2px 0;
  transition: transform var(--transition-fast) ease;
}

.sider-menu-item-hover:hover::before {
  transform: translateY(-50%) scaleY(1);
}

/* ========== 选中态背景覆盖 ========== */
.role-selected-overlay {
  background: linear-gradient(
    90deg,
    rgba(22, 119, 255, 0.1) 0%,
    transparent 100%
  ) !important;
}

/* ========== Logo 呼吸动画 ========== */
.logo-breathe {
  animation: breathe 3s ease-in-out infinite;
}
```

```
// ===== File: src\main.tsx =====
/**
 * 青宸智汇 React 子应用 - 入口文件
 * 支持 qiankun 微前端和独立运行两种模式
 */

import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App';
import AppProviders from './components/AppProviders';
import './index.css';

// ========== qiankun 子应用生命周期 ==========

let root: ReactDOM.Root | null = null;

/**
 * bootstrap - 只在微应用初始化时调用一次
 */
export async function bootstrap() {
  console.log('[青宸智汇 Chat] bootstrap');
}

/**
 * mount - 每次进入微应用时调用
 */
export async function mount(props: Record<string, unknown>) {
  console.log('[青宸智汇 Chat] mount', props);

  const container = (props.container as HTMLElement) || document.getElementById('root');
  if (!container) {
    console.error('[青宸智汇 Chat] mount failed: container not found');
    return;
  }

  // 创建挂载点
  let mountNode = container.querySelector('#ai-chat-root');
  if (!mountNode) {
    mountNode = document.createElement('div');
    mountNode.id = 'ai-chat-root';
    container.appendChild(mountNode);
  }

  root = ReactDOM.createRoot(mountNode);
  root.render(
    <React.StrictMode>
      <AppProviders>
        <App />
      </AppProviders>
    </React.StrictMode>
```

```
  );
}

/**
 * unmount - 每次离开微应用时调用
 */
export async function unmount() {
  console.log('[青宸智汇 Chat] unmount');
  if (root) {
    root.unmount();
    root = null;
  }

  // 清理挂载点
  const mountNode = document.getElementById('ai-chat-root');
  if (mountNode) {
    mountNode.remove();
  }
}

/**
 * update - 可选，主应用更新时调用
 */
export async function update(props: Record<string, unknown>) {
  console.log('[青宸智汇 Chat] update', props);
}

// ========== 独立运行模式 ==========
// 当不在 qiankun 环境中时，直接挂载到 #root
if (!(window as unknown as Record<string, unknown>).__POWERED_BY_QIANKUN__) {
  const rootNode = document.getElementById('root');
  if (rootNode) {
    root = ReactDOM.createRoot(rootNode);
    root.render(
      <React.StrictMode>
        <AppProviders>
          <App />
        </AppProviders>
      </React.StrictMode>
    );
  }
}

// ===== File: src\components\AIGeneratorForm.tsx =====
/**
 * 通用 AI 生成表单组件
 * 输入表单 + 生成按钮 + 结果展示
 * variant="sage" 启用军师AI 分区布局；variant="maker" 启用工匠AI 分区布局（默认保持通用样式）
 */
```

```
import React, { useState, useRef } from 'react';
import { Input, Button, Form, Spin, Typography, Card } from 'antd';
import { SendOutlined, DownloadOutlined } from '@ant-design/icons';
import { chatWithZhipu } from '../services/aiService';
import { useAIStore } from '../store/aiStore';
import { useI18n } from '../i18n';
import { SAGE_THEMES, SAGE_FONT_SERIF, type SageTheme, type SageThemeKey } from './sage/sage-theme';
import './sage/sage-animations.css';
import { SageSection } from './sage/shared';
import { MAKER_THEMES, MAKER_FONT_SERIF, type MakerTheme, type MakerThemeKey } from './maker/maker-theme';
import './maker/maker-animations.css';
import { MakerSection } from './maker/shared';

const { TextArea } = Input;
const { Title, Text } = Typography;

interface AIGeneratorFormProps {
  title: string;
  systemPrompt: string;
  fields: { name: string; label: string; placeholder: string; required?: boolean }[];
  resultTitle: string;
  generateLabel?: string;
  /** 'sage' 军师AI / 'maker' 工匠AI 分区布局；缺省为通用样式 */
  variant?: 'sage' | 'maker';
  /** 军师主题 key（variant="sage" 时生效） */
  sageTheme?: SageThemeKey;
  /** 工匠主题 key（variant="maker" 时生效） */
  makerTheme?: MakerThemeKey;
  /** 自定义结果渲染器：将 AI 返回的 Markdown 渲染为分区卡片 */
  resultRenderer?: (
    result: string,
    ctx: { theme: SageTheme; isDark: boolean; handleDownload: () => void }
  ) => React.ReactNode;
  /** 内置示例内容：提供时显示"查看示例"按钮，点击直接展示（无需调用 AI） */
  demoContent?: string;
  /** 示例按钮文字，默认"查看示例" */
  demoLabel?: string;
  /** 可选：生成成功后持久化到 localStorage 的 key（如 'ai-mate-sage-requirements-report'），不传则不持久化 */
  persistKey?: string;
}

const AIGeneratorForm: React.FC<AIGeneratorFormProps> = ({
  title,
  systemPrompt,
  fields,
  resultTitle,
  generateLabel,
  variant,
  sageTheme = 'requirements',
  makerTheme = 'ppt',
```

```
  resultRenderer,
  demoContent,
  demoLabel,
  persistKey,
}) => {
  const [form] = Form.useForm();
  const { t } = useI18n();
  const generateText = generateLabel ?? t('aiGen.generate');
  const demoText = demoLabel ?? t('aiGen.viewDemo');
  const [loading, setLoading] = useState(false);
  const [result, setResult] = useState('');
  const resultRef = useRef('');

  const isDark = useAIStore((s) => s.settings.theme === 'dark');
  const theme: SageTheme = SAGE_THEMES[sageTheme];

  const handleGenerate = async () => {
    const values = await form.validateFields();
    setLoading(true);
    setResult('');
    resultRef.current = '';

    const userContent = Object.entries(values)
      .map(([k, v]) => `${k}: ${v}`)
      .join('\n');

    try {
      const res = await chatWithZhipu(
        [{ role: 'user', content: userContent }],
        { system_prompt: systemPrompt }
      );
      const content = res.data?.choices?.[0]?.message?.content || '';
      setResult(content);
      if (persistKey) {
        try {
          localStorage.setItem(
            persistKey,
            JSON.stringify({ inputs: values, result: content, updatedAt: Date.now() })
          );
        } catch {
          // 静默失败
        }
      }
    } catch {
      setResult(t('aiGen.generateFailed'));
    } finally {
      setLoading(false);
    }
  };
```

```
const handleDownload = () => {
  const blob = new Blob([result], { type: 'text/markdown' });
  const url = URL.createObjectURL(blob);
  const a = document.createElement('a');
  a.href = url;
  a.download = `${title}.md`;
  a.click();
  URL.revokeObjectURL(url);
};

// ──── 军师AI 分区式渲染 ──────────────────────────────────────────────

if (variant === 'sage') {
  const textColor = isDark ? theme.textDark : theme.textLight;
  const borderColor = isDark ? theme.borderDark : theme.borderLight;

  return (
    <div
      className="sage-grid-bg sage-paper-noise"
      style={{
        padding: 16,
        background: isDark ? theme.bgDark : theme.bgLight,
        borderRadius: 12,
        minHeight: '100%',
        '--sage-grid-line': isDark ? theme.glowColor : 'rgba(120,100,60,0.05)',
      } as React.CSSProperties}
    >
      {/* 面板头部：案号徽章 + 标题 */}
      <div
        className="sage-fade-in-up"
        style={{
          display: 'flex',
          alignItems: 'center',
          gap: 12,
          padding: '14px 18px',
          borderRadius: 12,
          background: isDark ? theme.gradient : theme.gradientLight,
          marginBottom: 16,
          border: `1px solid ${borderColor}`,
        }}
      >
        <div
          style={{
            width: 44,
            height: 44,
            borderRadius: 8,
            background: theme.sealColor,
            color: '#fff',
            display: 'flex',
            alignItems: 'center',
```

```
        justifyContent: 'center',
        fontFamily: SAGE_FONT_SERIF,
        fontSize: 14,
        fontWeight: 700,
        letterSpacing: 2,
        boxShadow: `0 0 12px ${theme.glowColor}`,
        flexShrink: 0,
      }}
    >
      {theme.caseNo}
    </div>
    <div>
      <div
        style={{
          fontFamily: SAGE_FONT_SERIF,
          fontSize: 18,
          fontWeight: 700,
          color: isDark ? theme.textDark : theme.textLight,
          letterSpacing: 2,
        }}
      >
        {theme.title}
      </div>
      <div
        style={{
          fontFamily: SAGE_FONT_SERIF,
          fontSize: 11,
          color: theme.accentColor,
          letterSpacing: 3,
          opacity: 0.85,
        }}
      >
        STRATEGY SANDBOX
      </div>
    </div>
  </div>

  {/* 分区1：军情设定 */}
  <SageSection title={t('aiGen.intelSetup')} subtitle="INTEL SETUP" theme={theme} isDark={isDark} stagger={1}>
    <Form form={form} layout="vertical">
      {fields.map((f) => (
        <Form.Item
          key={f.name}
          name={f.name}
          label={
            <span style={{ fontFamily: SAGE_FONT_SERIF, color: textColor, fontSize: 13 }}>
              {f.label}
            </span>
          }
          rules={f.required !== false ? [{ required: true, message: t('aiGen.required', { label: f.label }) }] : [
```

```
    >
      {f.name.includes('desc') || f.name.includes('content') || f.name.includes('intro') ? (
        <TextArea
          rows={3}
          placeholder={f.placeholder}
          style={{
            background: isDark ? theme.surfaceDark : '#fff',
            borderColor: borderColor,
            color: textColor,
            borderRadius: 8,
            fontFamily: SAGE_FONT_SERIF,
          }}
        />
      ) : (
        <Input
          placeholder={f.placeholder}
          style={{
            background: isDark ? theme.surfaceDark : '#fff',
            borderColor: borderColor,
            color: textColor,
            borderRadius: 8,
            height: 36,
            fontFamily: SAGE_FONT_SERIF,
          }}
        />
      )}
    </Form.Item>
  ))}
  <Form.Item style={{ marginBottom: 0, display: 'flex', justifyContent: 'flex-end' }}>
    <Button
      className="sage-seal-btn"
      type="primary"
      icon={<SendOutlined />}
      onClick={handleGenerate}
      loading={loading}
      style={{
        background: theme.sealColor,
        border: 'none',
        borderRadius: 8,
        height: 40,
        paddingLeft: 24,
        paddingRight: 24,
        fontFamily: SAGE_FONT_SERIF,
        fontSize: 14,
        fontWeight: 700,
        letterSpacing: 3,
        boxShadow: `0 4px 14px ${theme.glowColor}`,
      }}
    >
      {generateText}
```

```
      </Button>
    </Form.Item>
  </Form>
</SageSection>

{/* 分区2：推演结果 */}
{result && (
  <div style={{ marginTop: 16 }}>
    {resultRenderer ? (
      resultRenderer(result, { theme, isDark, handleDownload })
    ) : (
      <SageSection
        title={resultTitle}
        subtitle="RESULT"
        theme={theme}
        isDark={isDark}
        stagger={2}
      >
        <div
          style={{
            display: 'flex',
            justifyContent: 'flex-end',
            marginBottom: 8,
          }}
        >
          <Button
            type="text"
            icon={<DownloadOutlined />}
            onClick={handleDownload}
            size="small"
            style={{
              color: theme.accentColor,
              fontFamily: SAGE_FONT_SERIF,
              fontSize: 12,
            }}
          >
            {t('aiGen.exportMarkdown')}
          </Button>
        </div>
        <div
          style={{
            background: isDark ? 'rgba(0,0,0,0.25)' : '#FAF6EF',
            borderRadius: 8,
            border: `1px solid ${borderColor}`,
            padding: '14px 16px',
            maxHeight: 420,
            overflow: 'auto',
          }}
        >
          <Text
```

```
                  style={{
                    color: textColor,
                    whiteSpace: 'pre-wrap',
                    fontFamily: SAGE_FONT_SERIF,
                    lineHeight: 1.9,
                    fontSize: 13.5,
                  }}
                >
                  {result}
                </Text>
              </div>
            </SageSection>
          )}
        </div>
      )}

      {/* 加载态 */}
      {loading && (
        <div style={{ textAlign: 'center', padding: 32 }}>
          <Spin />
          <Text style={{ display: 'block', marginTop: 12, color: textColor, fontFamily: SAGE_FONT_SERIF }}>
            {t('aiGen.sageLoading')}
          </Text>
        </div>
      )}
    </div>
  );
}

// ——— 工匠AI 分区式渲染 ————————————————————————————————————————————

if (variant === 'maker') {
  const mTheme: MakerTheme = MAKER_THEMES[makerTheme];
  const mText = isDark ? mTheme.textDark : mTheme.textLight;
  const mBorder = isDark ? mTheme.borderDark : mTheme.borderLight;

  return (
    <div
      className="maker-grid-bg maker-paper-noise"
      style={{
        padding: 16,
        background: isDark ? mTheme.bgDark : mTheme.bgLight,
        borderRadius: 12,
        minHeight: '100%',
        '--maker-grid-line': isDark ? mTheme.glowColor : 'rgba(90,90,80,0.05)',
      } as React.CSSProperties}
    >
      {/* 面板头部：案号徽章 + 标题 */}
      <div
        className="maker-fade-in-up"
```

```
        style={{
          display: 'flex',
          alignItems: 'center',
          gap: 12,
          padding: '14px 18px',
          borderRadius: 12,
          background: isDark ? mTheme.gradient : mTheme.gradientLight,
          marginBottom: 16,
          border: `1px solid ${mBorder}`,
        }}
      >
        <div
          style={{
            width: 44,
            height: 44,
            borderRadius: 8,
            background: mTheme.sealColor,
            color: '#fff',
            display: 'flex',
            alignItems: 'center',
            justifyContent: 'center',
            fontFamily: MAKER_FONT_SERIF,
            fontSize: 14,
            fontWeight: 700,
            letterSpacing: 2,
            boxShadow: `0 0 12px ${mTheme.glowColor}`,
            flexShrink: 0,
          }}
        >
          {mTheme.caseNo}
        </div>
        <div>
          <div
            style={{
              fontFamily: MAKER_FONT_SERIF,
              fontSize: 18,
              fontWeight: 700,
              color: isDark ? mTheme.textDark : mTheme.textLight,
              letterSpacing: 2,
            }}
          >
            {mTheme.title}
          </div>
          <div
            style={{
              fontFamily: MAKER_FONT_SERIF,
              fontSize: 11,
              color: mTheme.accentColor,
              letterSpacing: 3,
              opacity: 0.85,
```

```
        }}
      >
        MAKER WORKSHOP
      </div>
    </div>
  </div>

  {/* 分区1：设定 */}
  <MakerSection title={t('aiGen.makerSection')} subtitle="SETUP" theme={mTheme} isDark={isDark} stagger={1}>
    <Form form={form} layout="vertical">
      {fields.map((f) => (
        <Form.Item
          key={f.name}
          name={f.name}
          label={
            <span style={{ fontFamily: MAKER_FONT_SERIF, color: mText, fontSize: 13 }}>
              {f.label}
            </span>
          }
          rules={f.required !== false ? [{ required: true, message: t('aiGen.required', { label: f.label }) }] : [
        >
          {f.name.includes('desc') || f.name.includes('content') || f.name.includes('intro') ? (
            <TextArea
              rows={3}
              placeholder={f.placeholder}
              style={{
                background: isDark ? mTheme.surfaceDark : '#fff',
                borderColor: mBorder,
                color: mText,
                borderRadius: 8,
                fontFamily: MAKER_FONT_SERIF,
              }}
            />
          ) : (
            <Input
              placeholder={f.placeholder}
              style={{
                background: isDark ? mTheme.surfaceDark : '#fff',
                borderColor: mBorder,
                color: mText,
                borderRadius: 8,
                height: 36,
                fontFamily: MAKER_FONT_SERIF,
              }}
            />
          )}
        </Form.Item>
      ))}
      <Form.Item style={{ marginBottom: 0, display: 'flex', justifyContent: 'flex-end', gap: 10 }}>
        {demoContent && (
```

```
            <Button
              onClick={() => {
                setResult(demoContent);
                resultRef.current = demoContent;
              }}
              style={{
                borderColor: mTheme.accentColor,
                color: mTheme.accentColor,
                borderRadius: 8,
                height: 40,
                paddingLeft: 18,
                paddingRight: 18,
                fontFamily: MAKER_FONT_SERIF,
                fontSize: 13,
                fontWeight: 600,
              }}
            >
              {demoText}
            </Button>
          )}
          <Button
            className="maker-seal-btn"
            type="primary"
            icon={<SendOutlined />}
            onClick={handleGenerate}
            loading={loading}
            style={{
              background: mTheme.sealColor,
              border: 'none',
              borderRadius: 8,
              height: 40,
              paddingLeft: 24,
              paddingRight: 24,
              fontFamily: MAKER_FONT_SERIF,
              fontSize: 14,
              fontWeight: 700,
              letterSpacing: 3,
              boxShadow: `0 4px 14px ${mTheme.glowColor}`,
            }}
          >
            {generateText}
          </Button>
        </Form.Item>
      </Form>
    </MakerSection>

    {/* 分区2：结果 */}
    {result && (
      <div style={{ marginTop: 16 }}>
        {resultRenderer ? (
```

```
        resultRenderer(result, { theme: mTheme as unknown as SageTheme, isDark, handleDownload })
      ) : (
        <MakerSection title={resultTitle} subtitle="RESULT" theme={mTheme} isDark={isDark} stagger={2}>
          <div
            style={{
              display: 'flex',
              justifyContent: 'flex-end',
              marginBottom: 8,
            }}
          >
            <Button
              type="text"
              icon={<DownloadOutlined />}
              onClick={handleDownload}
              size="small"
              style={{
                color: mTheme.accentColor,
                fontFamily: MAKER_FONT_SERIF,
                fontSize: 12,
              }}
            >
              {t('aiGen.exportMarkdown')}
            </Button>
          </div>
          <div
            style={{
              background: isDark ? 'rgba(0,0,0,0.25)' : '#F7F7F5',
              borderRadius: 8,
              border: `1px solid ${mBorder}`,
              padding: '14px 16px',
              maxHeight: 420,
              overflow: 'auto',
            }}
          >
            <Text
              style={{
                color: mText,
                whiteSpace: 'pre-wrap',
                fontFamily: MAKER_FONT_SERIF,
                lineHeight: 1.9,
                fontSize: 13.5,
              }}
            >
              {result}
            </Text>
          </div>
        </MakerSection>
      )}
    </div>
  )}
```

```
      {/* 加载态 */}
      {loading && (
        <div style={{ textAlign: 'center', padding: 32 }}>
          <Spin />
          <Text style={{ display: 'block', marginTop: 12, color: mText, fontFamily: MAKER_FONT_SERIF }}>
            {t('aiGen.makerLoading')}
          </Text>
        </div>
      )}
    </div>
  );
}

// ——— 通用渲染（默认，maker/butler 保持原样） ————————————————

return (
  <div>
    <Title level={5} style={{ marginBottom: 16 }}>
      {title}
    </Title>
    <Form form={form} layout="vertical">
      {fields.map((f) => (
        <Form.Item
          key={f.name}
          name={f.name}
          label={f.label}
          rules={f.required !== false ? [{ required: true, message: t('aiGen.required', { label: f.label }) }] : []}
        >
          {f.name.includes('desc') || f.name.includes('content') || f.name.includes('intro') ? (
            <TextArea rows={3} placeholder={f.placeholder} />
          ) : (
            <Input placeholder={f.placeholder} />
          )}
        </Form.Item>
      ))}
      <Form.Item>
        <Button type="primary" icon={<SendOutlined />} onClick={handleGenerate} loading={loading}>
          {generateText}
        </Button>
      </Form.Item>
    </Form>

    {result && (
      <Card
        title={resultTitle}
        extra={
          <Button type="link" icon={<DownloadOutlined />} onClick={handleDownload}>
            {t('aiGen.exportMarkdown')}
          </Button>
```

```
          }
          style={{ marginTop: 16, background: '#f6ffed', borderColor: '#b7eb8f' }}
        >
          <Text style={{ whiteSpace: 'pre-wrap' }}>{result}</Text>
        </Card>
      )}
    </div>
  );
};

export default AIGeneratorForm;

// ===== File: src\components\AIRoleLayout.tsx =====
/**
 * 通用 AI 角色页面布局
 * 顶部标签切换子面板 + 下方内容区
 * 支持角色主题色动态注入 + 内容区进入动画
 */

import React, { useState, useEffect } from 'react';
import { Layout, Tabs, Typography, Avatar } from 'antd';
import type { AIRole } from '../store/aiStore';

const { Content } = Layout;
const { Title, Text } = Typography;

export interface PanelItem {
  key: string;
  label: string;
  children: React.ReactNode;
  fullHeight?: boolean;
}

interface AIRoleLayoutProps {
  role: AIRole;
  title: string;
  icon: React.ReactNode;
  description: string;
  panels: PanelItem[];
  themeColor?: string;
  themeSecondary?: string;
}

const ROLE_THEME_COLORS: Record<AIRole, { primary: string; secondary: string }> = {
  scout: { primary: '#1677ff', secondary: '#36cfc9' },
  sage: { primary: '#faad14', secondary: '#ffc53d' },
  maker: { primary: '#52c41a', secondary: '#95de64' },
  butler: { primary: '#eb2f96', secondary: '#ff85c0' },
};
```

```
const AIRoleLayout: React.FC<AIRoleLayoutProps> = ({
  title,
  icon,
  description,
  panels,
  role,
  themeColor,
  themeSecondary,
}) => {
  const [activeKey, setActiveKey] = useState(panels[0]?.key);
  const [animationKey, setAnimationKey] = useState(0);

  // 使用传入的颜色或根据角色获取默认颜色
  const colors = ROLE_THEME_COLORS[role];
  const primary = themeColor || colors.primary;
  const secondary = themeSecondary || colors.secondary;

  // Tab 切换时触发动画
  const handleTabChange = (key: string) => {
    setActiveKey(key);
    setAnimationKey((prev) => prev + 1);
  };

  // 动态注入角色色 CSS 变量
  useEffect(() => {
    document.documentElement.style.setProperty('--role-primary', primary);
    document.documentElement.style.setProperty('--role-secondary', secondary);
  }, [primary, secondary]);

  return (
    <Layout style={{ height: '100%', background: '#f5f5f5' }}>
      {/* 注入 Tabs 高度占满样式 + 角色色覆盖 */}
      <style>{`
        .ai-role-tabs .ant-tabs-content-holder {
          flex: 1 !important;
          min-height: 0 !important;
        }
        .ai-role-tabs .ant-tabs-content {
          height: 100% !important;
        }
        .ai-role-tabs .ant-tabs-tabpane {
          height: 100% !important;
        }
        .ai-role-tabs .ant-tabs-tab-active {
          position: relative;
        }
        .ai-role-tabs .ant-tabs-tab-active::after {
          content: '';
          position: absolute;
          bottom: 0;
```

```
},

_loadFromStorage: () => {
  const data = loadServersFromStorage();
  if (data) set({ servers: data });
},

// CRUD
addServer: (serverData) => {
  const newServer: MCPServer = {
    ...serverData,
    id: generateId(),
    status: 'disconnected',
    tools: (serverData as MCPServer).tools || [],
    createdAt: Date.now(),
    updatedAt: Date.now(),
  };
  set((draft) => ({ servers: [newServer, ...draft.servers] }));
  saveServersToStorage(get().servers);
},

addServerFromTemplate: (tpl) => {
  const configObj: Record<string, unknown> = {
    name: tpl.name,
    transport: tpl.transport,
    ...(tpl.command ? { command: tpl.command } : {}),
    ...(tpl.args ? { args: tpl.args } : {}),
    ...(tpl.url ? { url: tpl.url } : {}),
  };
  const newServer: MCPServer = {
    id: generateId(),
    name: tpl.name,
    transport: tpl.transport,
    command: tpl.command,
    args: tpl.args,
    url: tpl.url,
    env: tpl.env || {},
    status: 'disconnected',
    tools: tpl.tools,
    configJson: JSON.stringify(configObj, null, 2),
    createdAt: Date.now(),
    updatedAt: Date.now(),
  };
  set((draft) => ({ servers: [newServer, ...draft.servers] }));
  saveServersToStorage(get().servers);
},

updateServer: (id, patch) => {
  set((draft) => ({
    servers: draft.servers.map((s) =>
```

```
      s.id === id ? { ...s, ...patch, updatedAt: Date.now() } : s
    ),
  }));
  saveServersToStorage(get().servers);
},

removeServer: (id) => {
  set((draft) => ({ servers: draft.servers.filter((s) => s.id !== id) }));
  saveServersToStorage(get().servers);
},

duplicateServer: (id) => {
  const source = get().servers.find((s) => s.id === id);
  if (!source) return;
  const copy: MCPServer = {
    ...source,
    id: generateId(),
    name: `${source.name} (副本)`,
    status: 'disconnected',
    createdAt: Date.now(),
    updatedAt: Date.now(),
  };
  set((draft) => ({ servers: [copy, ...draft.servers] }));
  saveServersToStorage(get().servers);
},

// 连接状态
setServerStatus: (id, status) => {
  set((draft) => ({
    servers: draft.servers.map((s) =>
      s.id === id ? { ...s, status, updatedAt: Date.now() } : s
    ),
  }));
},

setServerTools: (id, tools) => {
  set((draft) => ({
    servers: draft.servers.map((s) =>
      s.id === id ? { ...s, tools, updatedAt: Date.now() } : s
    ),
  }));
  saveServersToStorage(get().servers);
},

// 测试连接（模拟异步，后续可替换为真实连接逻辑）
testConnection: async (id) => {
  const server = get().servers.find((s) => s.id === id);
  if (!server) return false;

  set((draft) => ({
```

```
      servers: draft.servers.map((s) =>
        s.id === id ? { ...s, status: 'connecting' as MCPStatus } : s
      ),
    }));

    // 模拟网络延迟
    await new Promise((resolve) => setTimeout(resolve, 1500));

    // 模拟 85% 成功率
    const success = Math.random() > 0.15;
    const newStatus: MCPStatus = success ? 'connected' : 'error';

    set((draft) => ({
      servers: draft.servers.map((s) =>
        s.id === id ? { ...s, status: newStatus, updatedAt: Date.now() } : s
      ),
    }));
    saveServersToStorage(get().servers);

    return success;
  },

  // JSON 导入
  importFromJson: (jsonString) => {
    const parsed = JSON.parse(jsonString);
    const serversToImport = Array.isArray(parsed) ? parsed : [parsed];

    serversToImport.forEach((s: Record<string, unknown>) => {
      const newServer: MCPServer = {
        id: generateId(),
        name: (s.name as string) || '未命名服务器',
        transport: (s.transport as 'stdio' | 'sse') || 'stdio',
        command: s.command as string | undefined,
        args: s.args as string[] | undefined,
        env: (s.env as Record<string, string>) || undefined,
        url: s.url as string | undefined,
        status: 'disconnected',
        tools: [],
        configJson: JSON.stringify(s, null, 2),
        createdAt: Date.now(),
        updatedAt: Date.now(),
      };
      set((draft) => ({ servers: [newServer, ...draft.servers] }));
    });
    saveServersToStorage(get().servers);
  },

  // JSON 导出（单个）
  exportToJson: (id) => {
    const server = get().servers.find((s) => s.id === id);
```

```
      if (!server) return '{}';
      return server.configJson;
    },

    // JSON 导出（全部）
    exportAllToJson: () => {
      return JSON.stringify(get().servers.map((s) => JSON.parse(s.configJson)), null, 2);
    },

    // 批量删除
    batchRemove: (ids) => {
      const idSet = new Set(ids);
      set((draft) => ({ servers: draft.servers.filter((s) => !idSet.has(s.id)) }));
      saveServersToStorage(get().servers);
    },
  };
});

// ===== File: src\store\notificationStore.ts =====
/**
 * 消息中心状态管理
 */
import { create } from 'zustand';
import {
  fetchNotifications,
  markNotificationRead,
  markAllNotificationsRead,
  clearAllNotifications,
  type NotificationItem,
} from '../services/notificationService';

interface NotificationStore {
  notifications: NotificationItem[];
  unreadCount: number;
  loading: boolean;
  drawerOpen: boolean;
  fetchAll: () => Promise<void>;
  markOne: (id: number) => Promise<void>;
  markAll: () => Promise<void>;
  clearAll: () => Promise<void>;
  open: () => void;
  close: () => void;
}

export const useNotificationStore = create<NotificationStore>((set) => ({
  notifications: [],
  unreadCount: 0,
  loading: false,
  drawerOpen: false,
```

```
  fetchAll: async () => {
    set({ loading: true });
    const data = await fetchNotifications(50);
    set({ notifications: data.list, unreadCount: data.unreadCount, loading: false });
  },

  markOne: async (id) => {
    const ok = await markNotificationRead(id);
    if (ok) {
      set((s) => ({
        notifications: s.notifications.map((n) => (n.id === id ? { ...n, isRead: true } : n)),
        unreadCount: Math.max(0, s.unreadCount - 1),
      }));
    }
  },

  markAll: async () => {
    const ok = await markAllNotificationsRead();
    if (ok) {
      set((s) => ({
        notifications: s.notifications.map((n) => ({ ...n, isRead: true })),
        unreadCount: 0,
      }));
    }
  },

  clearAll: async () => {
    const ok = await clearAllNotifications();
    if (ok) {
      set({ notifications: [], unreadCount: 0 });
    }
  },

  open: () => set({ drawerOpen: true }),
  close: () => set({ drawerOpen: false }),
}));

// ===== File: src\store\planStore.ts =====
/**
 * 计划模式状态管理 (Zustand)
 * 参考 EvoFlow Supervisor Plan 模式:
 *   Supervisor 澄清意图 → plan() 生成可审阅计划 → 用户授权 → 按 depends_on 派发子任务
 * 支持多角色 DAG 执行（探路者/军师/工匠/管家）。
 */

import { create } from 'zustand';
import { immer } from 'zustand/middleware/immer';
import type { WritableDraft } from 'immer';
import type { AIRole } from './aiStore';
```

```
export type PlanStepStatus = 'pending' | 'ready' | 'running' | 'completed' | 'failed';
export type PlanStatus = 'draft' | 'approved' | 'running' | 'completed' | 'failed' | 'cancelled';

export interface PlanStep {
  id: string;
  title: string;
  description: string;
  /** 执行角色：scout/sage/maker/butler */
  assignedRole: AIRole;
  /** 依赖的上游步骤 id 列表 */
  dependsOn: string[];
  /** 验收标准 */
  acceptance?: string;
  status: PlanStepStatus;
  result?: string;
  error?: string;
  startedAt?: number;
  finishedAt?: number;
}

export interface Plan {
  id: string;
  goal: string;
  steps: PlanStep[];
  status: PlanStatus;
  createdAt: number;
  updatedAt: number;
}

interface PlanStore {
  plans: Plan[];
  activePlanId: string | null;
  activeRole: AIRole;

  createPlan: (goal: string, steps: Omit<PlanStep, 'status'>[], role: AIRole) => string;
  approvePlan: (planId: string) => void;
  cancelPlan: (planId: string) => void;
  /** 将状态为 completed 的步骤依赖的待执行步骤推进为 ready */
  setStepStatus: (planId: string, stepId: string, status: PlanStepStatus, extra?: Partial<PlanStep>) => void;
  /** 计算当前可执行步骤（依赖全部 completed 且自身非终态） */
  getRunnableSteps: (planId: string) => PlanStep[];
  isPlanComplete: (planId: string) => boolean;
  deletePlan: (planId: string) => void;
  setActiveRole: (role: AIRole) => void;
}

const generateId = () => `${Date.now()}-${Math.random().toString(36).slice(2, 9)}`;

const ROLE_ORDER: Record<AIRole, number> = { scout: 1, sage: 2, maker: 3, butler: 4 };
```

```
// 依赖排序：按 depends_on 拓扑排序（简单 Kahn 算法），同层按角色顺序
const topoSort = (steps: PlanStep[]): PlanStep[] => {
  const ids = new Set(steps.map((s) => s.id));
  const result: PlanStep[] = [];
  const visited = new Set<string>();
  const visit = (step: PlanStep) => {
    if (visited.has(step.id)) return;
    visited.add(step.id);
    for (const dep of step.dependsOn) {
      const d = steps.find((s) => s.id === dep);
      if (d && !visited.has(dep)) visit(d);
    }
    result.push(step);
  };
  // 先按角色顺序访问，保证同层顺序稳定
  [...steps]
    .sort((a, b) => (ROLE_ORDER[a.assignedRole] || 99) - (ROLE_ORDER[b.assignedRole] || 99))
    .forEach(visit);
  return result;
};

export const usePlanStore = create<PlanStore>()(
  immer<PlanStore>((set, get) => ({
    plans: [],
    activePlanId: null,
    activeRole: 'scout',

    createPlan: (goal, steps, role) => {
      const id = generateId();
      const plan: Plan = {
        id,
        goal,
        steps: steps.map((s) => ({ ...s, status: 'pending' })),
        status: 'draft',
        createdAt: Date.now(),
        updatedAt: Date.now(),
      };
      set((draft: WritableDraft<PlanStore>) => {
        draft.plans.unshift(plan);
        draft.activePlanId = id;
        draft.activeRole = role;
      });
      return id;
    },

    approvePlan: (planId) => {
      set((draft: WritableDraft<PlanStore>) => {
        const plan = draft.plans.find((p) => p.id === planId);
        if (!plan || plan.status !== 'draft') return;
        plan.status = 'approved';
```

```
    plan.updatedAt = Date.now();
    // 无依赖的步骤置为 ready
    plan.steps.forEach((s) => {
      if (s.dependsOn.length === 0 && s.status === 'pending') s.status = 'ready';
    });
  });
},

cancelPlan: (planId) => {
  set((draft: WritableDraft<PlanStore>) => {
    const plan = draft.plans.find((p) => p.id === planId);
    if (!plan) return;
    plan.status = 'cancelled';
    plan.steps.forEach((s) => {
      if (s.status === 'pending' || s.status === 'ready' || s.status === 'running') {
        s.status = 'pending';
      }
    });
    plan.updatedAt = Date.now();
  });
},

setStepStatus: (planId, stepId, status, extra) => {
  set((draft: WritableDraft<PlanStore>) => {
    const plan = draft.plans.find((p) => p.id === planId);
    if (!plan) return;
    const step = plan.steps.find((s) => s.id === stepId);
    if (!step) return;
    step.status = status;
    if (extra) Object.assign(step, extra);
    if (status === 'running') step.startedAt = Date.now();
    if (status === 'completed' || status === 'failed') step.finishedAt = Date.now();
    plan.updatedAt = Date.now();

    // 步骤完成后推进依赖它的下游步骤
    if (status === 'completed') {
      for (const s of plan.steps) {
        if (s.status === 'pending' && s.dependsOn.length > 0) {
          const allDone = s.dependsOn.every(
            (dep) => plan.steps.find((d) => d.id === dep)?.status === 'completed'
          );
          if (allDone) s.status = 'ready';
        }
      }
      // 全部完成则计划完成
      if (plan.steps.every((s) => s.status === 'completed')) {
        plan.status = 'completed';
      }
    }
    if (status === 'failed') {
```

```
          plan.status = 'failed';
        }
      });
    },

    getRunnableSteps: (planId) => {
      const plan = get().plans.find((p) => p.id === planId);
      if (!plan) return [];
      const runnable = plan.steps.filter((s) => s.status === 'ready' || s.status === 'running');
      return topoSort(runnable);
    },

    isPlanComplete: (planId) => {
      const plan = get().plans.find((p) => p.id === planId);
      return !!plan && plan.steps.length > 0 && plan.steps.every((s) => s.status === 'completed');
    },

    deletePlan: (planId) => {
      set((draft: WritableDraft<PlanStore>) => {
        draft.plans = draft.plans.filter((p) => p.id !== planId);
        if (draft.activePlanId === planId) draft.activePlanId = null;
      });
    },

    setActiveRole: (role) => set({ activeRole: role }),
  }))
);

// ===== File: src\store\skillStore.ts =====
/**
 * Skill 库状态管理 (Zustand)
 * 参考 Grok Build Skills 系统设计
 */

import { create } from 'zustand';
import type { Skill, SkillCategory, AutoTrigger } from '../types';

interface SkillStore {
  // 数据
  skills: Skill[];

  // 筛选与搜索
  searchQuery: string;
  activeCategory: SkillCategory | 'all';
  setSearchQuery: (q: string) => void;
  setActiveCategory: (cat: SkillCategory | 'all') => void;

  // CRUD
  addSkill: (skill: Omit<Skill, 'id' | 'createdAt' | 'updatedAt' | 'usageCount'>) => void;
  updateSkill: (id: string, patch: Partial<Skill>) => void;
```

```
  deleteSkill: (id: string) => void;
  toggleSkill: (id: string) => void;

  // 使用统计
  incrementUsage: (id: string) => void;

  // 派生数据
  getFilteredSkills: () => Skill[];
}

const generateId = () => `${Date.now()}-${Math.random().toString(36).slice(2, 9)}`;

// 默认预设 Skills（参考 Grok Build 的 Skill 发现机制 + SkillHub & RedSkill 平台集成）
const defaultSkills: Skill[] = [
  // ===== 原生 Skills =====
  {
    id: generateId(),
    name: '创业市场调研',
    description: '自动分析目标市场规模、竞品格局和用户痛点',
    category: 'analysis',
    tags: ['analysis'],
    promptTemplate:
      '你是一位资深市场分析师。请针对"{{topic}}"进行深度市场调研，输出：\n1. 市场规模与增长趋势\n2. 核心竞品分析（3-5家）\n3. 目标用户画
    triggerCommand: '/market-research',
    autoTriggers: [{ id: generateId(), type: 'keyword', condition: '市场调研' }],
    isEnabled: true,
    usageCount: 0,
    createdAt: Date.now(),
    updatedAt: Date.now(),
  },
  {
    id: generateId(),
    name: 'BP 精炼助手',
    description: '将粗糙的商业想法提炼为结构清晰的 BP 大纲',
    category: 'writing',
    tags: ['writing'],
    promptTemplate:
      '你是一位 BP 撰写专家。请将以下商业想法精炼成一份投资人级别的 BP 大纲：\n\n{{idea}}\n\n要求包含：执行摘要、痛点、解决方案、商业模
    triggerCommand: '/bp-refine',
    autoTriggers: [],
    isEnabled: true,
    usageCount: 0,
    createdAt: Date.now(),
    updatedAt: Date.now(),
  },
  {
    id: generateId(),
    name: '代码审查',
    description: '审查代码质量、安全性和性能问题',
    category: 'coding',
```

```
    tags: ['coding'],
    promptTemplate:
      '你是一位资深技术负责人。请审查以下代码，从可读性、安全性、性能、最佳实践四个维度给出评分和改进建议：\n\n```\n{{code}}\n```',
    triggerCommand: '/code-review',
    autoTriggers: [{ id: generateId(), type: 'keyword', condition: 'review my code' }],
    isEnabled: true,
    usageCount: 0,
    createdAt: Date.now(),
    updatedAt: Date.now(),
  },
  {
    id: generateId(),
    name: '营销文案生成',
    description: '根据产品特点生成多平台营销文案',
    category: 'marketing',
    tags: ['marketing'],
    promptTemplate:
      '你是一位 growth hacker。请为以下产品生成营销文案：\n\n产品：{{product}}\n卖点：{{sellingPoints}}\n\n输出：\n1. 小红书风格（emoji
    triggerCommand: '/marketing-copy',
    autoTriggers: [],
    isEnabled: true,
    usageCount: 0,
    createdAt: Date.now(),
    updatedAt: Date.now(),
  },

  // ===== SkillHub 平台 Skills =====
  // -- 知识管理 --
  {
    id: generateId(),
    name: 'AnySearch 实时搜索',
    description: '支持 Web 搜索、垂直搜索、批量并行搜索与 URL 内容提取',
    category: 'knowledge',
    tags: ['knowledge'],
    promptTemplate:
      '你是一位信息检索专家。请针对查询"{{query}}"执行以下任务：\n1. 分析用户查询意图\n2. 给出结构化搜索结果摘要\n3. 推荐 3-5 个最相关的
    triggerCommand: '/anysearch',
    autoTriggers: [{ id: generateId(), type: 'keyword', condition: '搜索' }],
    isEnabled: true,
    usageCount: 0,
    createdAt: Date.now(),
    updatedAt: Date.now(),
  },
  {
    id: generateId(),
    name: '全球文献检索',
    description: '覆盖 8000 万中文期刊与 12 亿全球文献元数据，支持三级检索',
    category: 'knowledge',
    tags: ['knowledge'],
    promptTemplate:
```

```
    '你是一位学术文献专家。请针对研究主题"{{topic}}"进行文献检索分析：\n1. 推荐 5-10 篇核心文献（含标题、作者、摘要）\n2. 梳理该领域的
  triggerCommand: '/literature',
  autoTriggers: [],
  isEnabled: true,
  usageCount: 0,
  createdAt: Date.now(),
  updatedAt: Date.now(),
},
// -- 办公效率 --
{
  id: generateId(),
  name: '智能 PPT 生成',
  description: '多行业多风格 PPT 生成，支持中英文双语与自动配色',
  category: 'office',
  tags: ['office'],
  promptTemplate:
    '你是一位专业 PPT 设计师。请为"{{topic}}"生成一份完整的 PPT 大纲：\n1. 封面标题与副标题\n2. 目录页结构\n3. 每页的内容要点（3-5 个
  triggerCommand: '/ppt-gen',
  autoTriggers: [{ id: generateId(), type: 'keyword', condition: 'PPT' }],
  isEnabled: true,
  usageCount: 0,
  createdAt: Date.now(),
  updatedAt: Date.now(),
},
{
  id: generateId(),
  name: 'OCR 文字提取',
  description: '多格式图片与 PDF 文字提取，保留原始格式',
  category: 'office',
  tags: ['office'],
  promptTemplate:
    '你是一位文档处理专家。用户上传了图片/PDF，请：\n1. 提取所有可见文字内容\n2. 保留原文的段落结构和格式\n3. 对模糊或不确定的文字标注
  triggerCommand: '/ocr',
  autoTriggers: [],
  isEnabled: true,
  usageCount: 0,
  createdAt: Date.now(),
  updatedAt: Date.now(),
},
// -- 内容创作 --
{
  id: generateId(),
  name: '文章去 AI 味',
  description: '去除文本 AI 写作痕迹，修复高频词与机械化表达',
  category: 'marketing',
  tags: ['marketing'],
  promptTemplate:
    '你是一位资深编辑。请将以下文本进行"去 AI 化"改写：\n\n{{text}}\n\n要求：\n1. 替换 AI 高频词汇（如" delve into", " embark on"等）
  triggerCommand: '/de-ai',
  autoTriggers: [{ id: generateId(), type: 'keyword', condition: '去AI' }],
```

```
    isEnabled: true,
    usageCount: 0,
    createdAt: Date.now(),
    updatedAt: Date.now(),
  },
  {
    id: generateId(),
    name: '短视频爆款拆解',
    description: '一键拆解爆款视频结构，输出分段、归因、六维评分',
    category: 'marketing',
    tags: ['marketing'],
    promptTemplate:
      '你是一位短视频运营专家。请对以下视频/链接进行爆款拆解：\n\n{{videoInfo}}\n\n输出：\n1. 视频结构分段（黄金 3 秒、中间、结尾）\n2.
    triggerCommand: '/video-deconstruct',
    autoTriggers: [],
    isEnabled: true,
    usageCount: 0,
    createdAt: Date.now(),
    updatedAt: Date.now(),
  },
  {
    id: generateId(),
    name: '抖音文案提取',
    description: '从抖音/快手/小红书/视频号链接提取标题、简介、口播文案',
    category: 'marketing',
    tags: ['marketing'],
    promptTemplate:
      '你是一位内容分析专家。请分析以下链接/文案：\n\n{{link}}\n\n输出：\n1. 原始标题与简介\n2. 口播文案全文\n3. 文案结构分析（钩子、正
    triggerCommand: '/extract-copy',
    autoTriggers: [],
    isEnabled: true,
    usageCount: 0,
    createdAt: Date.now(),
    updatedAt: Date.now(),
  },
  {
    id: generateId(),
    name: '番茄小说写作',
    description: '番茄小说平台分章节创作，支持多题材长篇创作',
    category: 'writing',
    tags: ['writing'],
    promptTemplate:
      '你是一位网文作家。请根据以下设定创作小说章节：\n\n题材：{{genre}}\n章节主题：{{theme}}\n字数要求：2200-2800 字\n\n要求：\n1. 节奏
    triggerCommand: '/novel-write',
    autoTriggers: [],
    isEnabled: true,
    usageCount: 0,
    createdAt: Date.now(),
    updatedAt: Date.now(),
  },
```

```
// -- 设计多媒体 --
{
  id: generateId(),
  name: '架构图生成',
  description: '纯文本驱动架构图生成，支持导出 PPT/SVG',
  category: 'design',
  tags: ['design'],
  promptTemplate:
    '你是一位技术架构师。请为"{{system}}"设计系统架构图：\n1. 用文本符号（ASCII/缩进）绘制架构图\n2. 标注各模块职责与交互关系\n3. 说明
  triggerCommand: '/arch-diagram',
  autoTriggers: [{ id: generateId(), type: 'keyword', condition: '架构图' }],
  isEnabled: true,
  usageCount: 0,
  createdAt: Date.now(),
  updatedAt: Date.now(),
},
{
  id: generateId(),
  name: '海报设计',
  description: '基于设计哲学生成可渲染 SVG 海报',
  category: 'design',
  tags: ['design'],
  promptTemplate:
    '你是一位平面设计师。请为"{{theme}}"设计海报：\n1. 海报概念与视觉主题\n2. 配色方案（主色、辅色、点缀色）\n3. 版式布局描述\n4. 文案
  triggerCommand: '/poster',
  autoTriggers: [],
  isEnabled: true,
  usageCount: 0,
  createdAt: Date.now(),
  updatedAt: Date.now(),
},
// -- 行业专业 --
{
  id: generateId(),
  name: '股票价值投资分析',
  description: 'A 股/港股价值投资分析，护城河、财务健康、DCF 估值',
  category: 'finance',
  tags: ['finance'],
  promptTemplate:
    '你是一位价值投资分析师。请对"{{stock}}"进行深度分析：\n1. 护城河分析（品牌、成本、网络效应）\n2. 财务健康度（ROE、现金流、负债率
  triggerCommand: '/stock-analysis',
  autoTriggers: [{ id: generateId(), type: 'keyword', condition: '股票' }],
  isEnabled: true,
  usageCount: 0,
  createdAt: Date.now(),
  updatedAt: Date.now(),
},
// -- 商业运营 --
{
  id: generateId(),
```

```
  name: '产品经理综合技能',
  description: '覆盖需求分析、PRD、KANO、SWOT、商业模式画布、OKR',
  category: 'product',
  tags: ['product'],
  promptTemplate:
    '你是一位产品总监。请针对"{{product}}"进行产品分析：\n1. 需求分析（用户痛点、场景、价值）\n2. PRD 核心要点\n3. KANO 模型分类\n4.
  triggerCommand: '/pm-master',
  autoTriggers: [{ id: generateId(), type: 'keyword', condition: '产品经理' }],
  isEnabled: true,
  usageCount: 0,
  createdAt: Date.now(),
  updatedAt: Date.now(),
},

// ===== RedSkill 平台 Skills（小红书生态） =====
{
  id: generateId(),
  name: '小红书深度研究',
  description: '小红书账号/内容/竞品的深度研究分析',
  category: 'analysis',
  tags: ['analysis'],
  promptTemplate:
    '你是一位小红书运营专家。请对"{{account}}"进行深度研究：\n1. 账号定位与内容策略分析\n2. 爆款笔记拆解（选题、封面、文案、标签）\n3
  triggerCommand: '/xhs-research',
  autoTriggers: [{ id: generateId(), type: 'keyword', condition: '小红书' }],
  isEnabled: true,
  usageCount: 0,
  createdAt: Date.now(),
  updatedAt: Date.now(),
},
{
  id: generateId(),
  name: '小红书内容发布',
  description: '小红书笔记内容生成与发布优化',
  category: 'marketing',
  tags: ['marketing'],
  promptTemplate:
    '你是一位小红书内容创作者。请为"{{topic}}"创作一篇小红书笔记：\n1. 吸睛标题（3 个备选）\n2. 正文内容（emoji 点缀、口语化）\n3. 标
  triggerCommand: '/xhs-publish',
  autoTriggers: [{ id: generateId(), type: 'keyword', condition: '小红书笔记' }],
  isEnabled: true,
  usageCount: 0,
  createdAt: Date.now(),
  updatedAt: Date.now(),
},
{
  id: generateId(),
  name: '小红书 MCP 集成',
  description: '通过 MCP 协议与小红书 API 交互，自动化运营',
  category: 'automation',
```

```
    tags: ['automation'],
    promptTemplate:
      '你是一位自动化运营专家。用户希望使用 MCP 协议操作小红书，请：\n1. 说明 MCP 连接配置步骤\n2. 列出可用的 API 操作（发布、查询、互
    triggerCommand: '/xhs-mcp',
    autoTriggers: [],
    isEnabled: true,
    usageCount: 0,
    createdAt: Date.now(),
    updatedAt: Date.now(),
  },
  {
    id: generateId(),
    name: '小红书图片卡片生成',
    description: '小红书封面图、笔记卡片与信息图自动化设计',
    category: 'design',
    tags: ['design'],
    promptTemplate:
      '你是一位小红书视觉设计师。请为"{{topic}}"设计笔记视觉：\n1. 封面图概念与配色\n2. 内页配图建议（3-5 张）\n3. 信息图布局方案\n4. 
    triggerCommand: '/xhs-design',
    autoTriggers: [],
    isEnabled: true,
    usageCount: 0,
    createdAt: Date.now(),
    updatedAt: Date.now(),
  },
  {
    id: generateId(),
    name: '小红书数据监控',
    description: '小红书账号数据监控与竞品分析',
    category: 'analysis',
    tags: ['analysis'],
    promptTemplate:
      '你是一位数据分析师。请针对小红书账号"{{account}}"输出数据报告：\n1. 粉丝增长趋势\n2. 内容互动率分析\n3. 爆款笔记特征总结\n4. 竞品
    triggerCommand: '/xhs-data',
    autoTriggers: [],
    isEnabled: true,
    usageCount: 0,
    createdAt: Date.now(),
    updatedAt: Date.now(),
  },
  {
    id: generateId(),
    name: '小红书评论互动',
    description: '评论自动化回复策略与互动数据分析',
    category: 'automation',
    tags: ['automation'],
    promptTemplate:
      '你是一位社区运营专家。请为小红书笔记制定互动策略：\n1. 评论区引导话术模板\n2. 高频问题自动回复方案\n3. 互动数据分析框架\n4. 用户
    triggerCommand: '/xhs-interact',
    autoTriggers: [],
```

```
    isEnabled: true,
    usageCount: 0,
    createdAt: Date.now(),
    updatedAt: Date.now(),
  },

  // ===== 扩展 Skills（SkillHub 高频） =====
  {
    id: generateId(),
    name: '公众号热门文章查询',
    description: '公众号爆款文章搜索与推荐工具',
    category: 'knowledge',
    tags: ['knowledge'],
    promptTemplate:
      '你是一位内容运营专家。请针对"{{topic}}"查询并推荐公众号热门文章：\n1. 推荐 3-5 篇相关爆款文章（标题、公众号、核心观点）\n2. 分析
    triggerCommand: '/gzh-search',
    autoTriggers: [{ id: generateId(), type: 'keyword', condition: '公众号' }],
    isEnabled: true,
    usageCount: 0,
    createdAt: Date.now(),
    updatedAt: Date.now(),
  },
  {
    id: generateId(),
    name: '腾讯文档协作',
    description: '腾讯文档官方技能，支持在线文档创建/编辑/读取/搜索/保存',
    category: 'office',
    tags: ['office'],
    promptTemplate:
      '你是一位文档协作专家。用户需要使用腾讯文档处理"{{topic}}"相关文档，请：\n1. 给出文档结构建议\n2. 提供协作流程规范\n3. 推荐模板与
    triggerCommand: '/tx-docs',
    autoTriggers: [],
    isEnabled: true,
    usageCount: 0,
    createdAt: Date.now(),
    updatedAt: Date.now(),
  },
  {
    id: generateId(),
    name: '金山文档协作',
    description: '金山文档（WPS/365.kdocs.cn）官方技能，支持云端文档新建/读取/编辑/搜索/分享',
    category: 'office',
    tags: ['office'],
    promptTemplate:
      '你是一位 WPS 办公专家。用户需要使用金山文档处理"{{topic}}"相关文档，请：\n1. 给出文档结构建议\n2. 提供协作流程规范\n3. 推荐 WPS
    triggerCommand: '/wps-docs',
    autoTriggers: [],
    isEnabled: true,
    usageCount: 0,
    createdAt: Date.now(),
```

```
    updatedAt: Date.now(),
  },
  {
    id: generateId(),
    name: '股票舆情智能助手',
    description: '美股行情、期货市场、财联社快讯、宏观国际、资金面与智能结论',
    category: 'finance',
    tags: ['finance'],
    promptTemplate:
      '你是一位金融舆情分析师。请针对"{{stock}}"进行舆情与资金面分析：\n1. 最新市场消息与机构观点汇总\n2. 资金流向与技术面简评\n3. 宏观
    triggerCommand: '/stock-sentiment',
    autoTriggers: [{ id: generateId(), type: 'keyword', condition: '舆情' }],
    isEnabled: true,
    usageCount: 0,
    createdAt: Date.now(),
    updatedAt: Date.now(),
  },
  {
    id: generateId(),
    name: 'Agently Mail AI Agent',
    description: 'QQ 邮箱团队为 Agent 打造的专属邮箱服务，支持读写邮件、搜索、回复、转发',
    category: 'automation',
    tags: ['automation'],
    promptTemplate:
      '你是一位邮件处理专家。用户需要处理邮件任务：{{task}}\n\n请：\n1. 给出邮件撰写/回复模板\n2. 提供邮件礼仪与沟通策略建议\n3. 说明自
    triggerCommand: '/mail-agent',
    autoTriggers: [{ id: generateId(), type: 'keyword', condition: '邮件' }],
    isEnabled: true,
    usageCount: 0,
    createdAt: Date.now(),
    updatedAt: Date.now(),
  },
  {
    id: generateId(),
    name: '视频号爆款文案生成',
    description: '免费口播脚本生成，按爆款结构产出口播文案',
    category: 'marketing',
    tags: ['marketing'],
    promptTemplate:
      '你是一位短视频文案专家。请为"{{topic}}"生成视频号口播文案：\n1. 黄金 3 秒钩子（3 个备选）\n2. 口播正文（150-300 字）\n3. 结尾 CT
    triggerCommand: '/sph-copy',
    autoTriggers: [{ id: generateId(), type: 'keyword', condition: '视频号' }],
    isEnabled: true,
    usageCount: 0,
    createdAt: Date.now(),
    updatedAt: Date.now(),
  },
  {
    id: generateId(),
    name: '爆款内容预检',
```

```
    description: '敏感词/广告法检测、模拟观众反应与爆款概率预测',
    category: 'marketing',
    tags: ['marketing'],
    promptTemplate:
      '你是一位内容审核专家。请对以下内容进行爆款预检：\n\n{{content}}\n\n输出：\n1. 敏感词与广告法风险检测\n2. 模拟观众反应（正面/负面
    triggerCommand: '/content-check',
    autoTriggers: [{ id: generateId(), type: 'keyword', condition: '预检' }],
    isEnabled: true,
    usageCount: 0,
    createdAt: Date.now(),
    updatedAt: Date.now(),
  },
  {
    id: generateId(),
    name: '视频号挂车爆单脚本',
    description: '基于视频号链接生成挂车脚本或原创多条口播脚本',
    category: 'marketing',
    tags: ['marketing'],
    promptTemplate:
      '你是一位电商带货文案专家。请为"{{product}}"生成视频号挂车爆单脚本：\n1. 产品痛点引入（3 秒钩子）\n2. 卖点拆解与使用场景展示\n3.
    triggerCommand: '/sph-sales',
    autoTriggers: [],
    isEnabled: true,
    usageCount: 0,
    createdAt: Date.now(),
    updatedAt: Date.now(),
  },
  {
    id: generateId(),
    name: '小红书视频下载分析',
    description: '小红书视频下载与解析，视频内容分析与结构化提取',
    category: 'automation',
    tags: ['automation'],
    promptTemplate:
      '你是一位视频分析专家。用户提供了小红书视频/链接，请：\n1. 提取视频核心内容与结构\n2. 分析画面节奏与剪辑技巧\n3. 总结口播文案与字
    triggerCommand: '/xhs-video',
    autoTriggers: [],
    isEnabled: true,
    usageCount: 0,
    createdAt: Date.now(),
    updatedAt: Date.now(),
  },
  {
    id: generateId(),
    name: 'Web 工具前置指南',
    description: '网页搜索/抓取前置知识管理技能，包含错误处理与 fallback 流程',
    category: 'knowledge',
    tags: ['knowledge'],
    promptTemplate:
      '你是一位网络爬虫与数据获取专家。用户需要获取"{{topic}}"相关网页信息，请：\n1. 给出最佳信息源与搜索策略\n2. 提供网页抓取的技术方案
```

```
    triggerCommand: '/web-tools',
    autoTriggers: [{ id: generateId(), type: 'keyword', condition: '网页抓取' }],
    isEnabled: true,
    usageCount: 0,
    createdAt: Date.now(),
    updatedAt: Date.now(),
  },

  // ===== 创业与网站搭建扩展 Skills =====
  {
    id: generateId(),
    name: '商业模式画布生成',
    description: '基于精益创业理念自动生成商业模式画布（BMC）',
    category: 'analysis',
    tags: ['analysis'],
    promptTemplate:
      '你是一位创业战略顾问。请为"{{idea}}"生成完整的商业模式画布：\n1. 客户细分（CS）\n2. 价值主张（VP）\n3. 渠道通路（CH）\n4. 客户关
    triggerCommand: '/business-model',
    autoTriggers: [{ id: generateId(), type: 'keyword', condition: '商业模式' }],
    isEnabled: true,
    usageCount: 0,
    createdAt: Date.now(),
    updatedAt: Date.now(),
  },
  {
    id: generateId(),
    name: '融资路演脚本',
    description: '生成投资人路演演讲稿与 PPT 结构，包含黄金 3 分钟钩子',
    category: 'writing',
    tags: ['writing'],
    promptTemplate:
      '你是一位资深 FA 与路演教练。请为"{{project}}"生成融资路演脚本：\n1. 黄金 3 分钟开场钩子\n2. 痛点与机会（Why Now）\n3. 解决方案与
    triggerCommand: '/pitch-deck',
    autoTriggers: [{ id: generateId(), type: 'keyword', condition: '路演' }],
    isEnabled: true,
    usageCount: 0,
    createdAt: Date.now(),
    updatedAt: Date.now(),
  },
  {
    id: generateId(),
    name: '股权架构设计',
    description: '创业公司股权分配、期权池设计与合伙人股权协议建议',
    category: 'finance',
    tags: ['finance'],
    promptTemplate:
      '你是一位股权设计专家。请针对"{{company}}"给出股权架构方案：\n1. 创始人股权分配原则与比例建议\n2. 期权池（ESOP）设定（10%-20%）\n
    triggerCommand: '/equity-design',
    autoTriggers: [{ id: generateId(), type: 'keyword', condition: '股权' }],
    isEnabled: true,
```

```
    usageCount: 0,
    createdAt: Date.now(),
    updatedAt: Date.now(),
  },
  {
    id: generateId(),
    name: 'MVP 设计与验证',
    description: '最小可行性产品设计、验证方案与 pivot 策略',
    category: 'product',
    tags: ['product'],
    promptTemplate:
      '你是一位精益创业导师。请为"{{idea}}"设计 MVP 方案：\n1. 核心假设提炼（价值假设/增长假设）\n2. MVP 功能清单（必须/最好/将来）\n3.
    triggerCommand: '/mvp-design',
    autoTriggers: [{ id: generateId(), type: 'keyword', condition: 'MVP' }],
    isEnabled: true,
    usageCount: 0,
    createdAt: Date.now(),
    updatedAt: Date.now(),
  },
  {
    id: generateId(),
    name: '创业法律合规检查',
    description: '公司注册、知识产权、数据合规、劳动合同等法律风险扫描',
    category: 'analysis',
    tags: ['analysis'],
    promptTemplate:
      '你是一位专注互联网创业的法律顾问。请针对"{{business}}"进行法律合规检查：\n1. 公司主体选择与注册地建议\n2. 知识产权布局（商标/专利
    triggerCommand: '/legal-check',
    autoTriggers: [{ id: generateId(), type: 'keyword', condition: '法律合规' }],
    isEnabled: true,
    usageCount: 0,
    createdAt: Date.now(),
    updatedAt: Date.now(),
  },
  {
    id: generateId(),
    name: '投资人 Pitch 邮件',
    description: '生成 cold email、跟进邮件与 TS 谈判邮件模板',
    category: 'writing',
    tags: ['writing'],
    promptTemplate:
      '你是一位融资顾问。请为"{{project}}"撰写投资人沟通邮件：\n1. Cold Email（标题 + 正文 + 附件清单）\n2. 首次会面后的跟进邮件\n3. 收
    triggerCommand: '/investor-email',
    autoTriggers: [{ id: generateId(), type: 'keyword', condition: '投资人' }],
    isEnabled: true,
    usageCount: 0,
    createdAt: Date.now(),
    updatedAt: Date.now(),
  },
  {
```

```
    id: generateId(),
    name: '网站技术选型指南',
    description: '前端框架、后端语言、数据库、云服务选型与架构建议',
    category: 'coding',
    tags: ['coding'],
    promptTemplate:
      '你是一位全栈架构师。用户要搭建"{{projectType}}"网站，请给出技术选型方案：\n1. 前端框架对比与推荐（React/Vue/Next/Nuxt）\n2. 后端
    triggerCommand: '/tech-stack',
    autoTriggers: [{ id: generateId(), type: 'keyword', condition: '技术选型' }],
    isEnabled: true,
    usageCount: 0,
    createdAt: Date.now(),
    updatedAt: Date.now(),
  },
  {
    id: generateId(),
    name: '域名与服务器选购',
    description: '域名注册、DNS 配置、服务器/VPS/云主机选购与备案准备',
    category: 'coding',
    tags: ['coding'],
    promptTemplate:
      '你是一位运维与基础设施专家。用户需要为"{{project}}"选购域名与服务器，请：\n1. 域名命名建议与注册平台对比\n2. 服务器类型选择（共享
    triggerCommand: '/domain-server',
    autoTriggers: [{ id: generateId(), type: 'keyword', condition: '域名' }],
    isEnabled: true,
    usageCount: 0,
    createdAt: Date.now(),
    updatedAt: Date.now(),
  },
  {
    id: generateId(),
    name: 'ICP 备案流程助手',
    description: '中国大陆 ICP 备案全流程指导，含材料模板与常见驳回原因',
    category: 'knowledge',
    tags: ['knowledge'],
    promptTemplate:
      '你是一位备案顾问。请针对"{{website}}"提供 ICP 备案全流程指导：\n1. 备案类型判断（个人/企业/经营性）\n2. 所需材料清单与模板下载建
    triggerCommand: '/icp-guide',
    autoTriggers: [{ id: generateId(), type: 'keyword', condition: '备案' }],
    isEnabled: true,
    usageCount: 0,
    createdAt: Date.now(),
    updatedAt: Date.now(),
  },
  {
    id: generateId(),
    name: '网站 SEO 优化方案',
    description: '站内优化、关键词策略、外链建设与搜索引擎收录加速',
    category: 'marketing',
    tags: ['marketing'],
```

```
    promptTemplate:
      '你是一位 SEO 专家。请为"{{website}}"制定 SEO 优化方案：\n1. 关键词研究与布局（核心词/长尾词/LSI）\n2. 站内优化（TDK/结构/速度/移
    triggerCommand: '/seo-optimize',
    autoTriggers: [{ id: generateId(), type: 'keyword', condition: 'SEO' }],
    isEnabled: true,
    usageCount: 0,
    createdAt: Date.now(),
    updatedAt: Date.now(),
  },
  {
    id: generateId(),
    name: '网站部署与 CI/CD',
    description: '自动化部署流水线、Docker 容器化、GitHub Actions 配置',
    category: 'automation',
    tags: ['automation'],
    promptTemplate:
      '你是一位 DevOps 工程师。请为"{{project}}"设计部署与 CI/CD 方案：\n1. 环境划分（开发/测试/生产）\n2. Docker 容器化与镜像优化\n3.
    triggerCommand: '/deploy-cicd',
    autoTriggers: [{ id: generateId(), type: 'keyword', condition: '部署' }],
    isEnabled: true,
    usageCount: 0,
    createdAt: Date.now(),
    updatedAt: Date.now(),
  },
  {
    id: generateId(),
    name: '前端性能优化审查',
    description: 'Lighthouse 指标分析、资源加载优化、渲染性能与缓存策略',
    category: 'coding',
    tags: ['coding'],
    promptTemplate:
      '你是一位前端性能专家。请审查以下网站/代码的性能问题：\n\n{{codeOrUrl}}\n\n输出：\n1. Lighthouse 核心指标评估（LCP/FID/CLS/TTFB）
    triggerCommand: '/frontend-perf',
    autoTriggers: [{ id: generateId(), type: 'keyword', condition: '性能优化' }],
    isEnabled: true,
    usageCount: 0,
    createdAt: Date.now(),
    updatedAt: Date.now(),
  },
];

const STORAGE_KEY = 'ai_mate_skills_v1';

const loadSkillsFromStorage = (): Skill[] | null => {
  try {
    const raw = localStorage.getItem(STORAGE_KEY);
    if (raw) {
      const parsed = JSON.parse(raw) as Skill[];
      // 兼容性：为旧数据补充 tags 字段
      return parsed.map((s) => ({
```

```
        ...s,
        tags: s.tags || [s.category],
      }));
    }
  } catch {}
  return null;
};

const saveSkillsToStorage = (skills: Skill[]) => {
  try {
    localStorage.setItem(STORAGE_KEY, JSON.stringify(skills));
  } catch {}
};

export const useSkillStore = create<SkillStore>((set, get) => ({
  // 初始数据：优先从 localStorage 加载，否则用默认
  skills: loadSkillsFromStorage() || defaultSkills,

  // 筛选与搜索
  searchQuery: '',
  activeCategory: 'all',
  setSearchQuery: (q) => set({ searchQuery: q }),
  setActiveCategory: (cat) => set({ activeCategory: cat }),

  // CRUD
  addSkill: (skillData) => {
    const newSkill: Skill = {
      ...skillData,
      id: generateId(),
      usageCount: 0,
      createdAt: Date.now(),
      updatedAt: Date.now(),
    };
    const nextSkills = [newSkill, ...get().skills];
    saveSkillsToStorage(nextSkills);
    set({ skills: nextSkills });
  },

  updateSkill: (id, patch) => {
    const nextSkills = get().skills.map((s) =>
      s.id === id ? { ...s, ...patch, updatedAt: Date.now() } : s
    );
    saveSkillsToStorage(nextSkills);
    set({ skills: nextSkills });
  },

  deleteSkill: (id) => {
    const nextSkills = get().skills.filter((s) => s.id !== id);
    saveSkillsToStorage(nextSkills);
    set({ skills: nextSkills });
```

```
  },

  toggleSkill: (id) => {
    const nextSkills = get().skills.map((s) =>
      s.id === id ? { ...s, isEnabled: !s.isEnabled, updatedAt: Date.now() } : s
    );
    saveSkillsToStorage(nextSkills);
    set({ skills: nextSkills });
  },

  // 使用统计
  incrementUsage: (id) => {
    const nextSkills = get().skills.map((s) =>
      s.id === id ? { ...s, usageCount: s.usageCount + 1, updatedAt: Date.now() } : s
    );
    saveSkillsToStorage(nextSkills);
    set({ skills: nextSkills });
  },

  // 派生数据
  getFilteredSkills: () => {
    const { skills, searchQuery, activeCategory } = get();
    return skills.filter((s) => {
      const matchCategory = activeCategory === 'all' || s.category === activeCategory;
      const matchSearch =
        !searchQuery ||
        s.name.toLowerCase().includes(searchQuery.toLowerCase()) ||
        s.description.toLowerCase().includes(searchQuery.toLowerCase()) ||
        s.triggerCommand.toLowerCase().includes(searchQuery.toLowerCase());
      return matchCategory && matchSearch;
    });
  },
}));

// ===== File: src\test\sample.test.ts =====
 import { describe, it, expect } from 'vitest'

 describe('vitest setup', () => {
   it('should run basic math', () => {
     expect(1 + 1).toBe(2)
   })
 })

// ===== File: src\test\setup.ts =====
 import '@testing-library/jest-dom'

// ===== File: src\types\index.ts =====
/**
 * 全局类型定义
 */
```

```
export type AIRole = 'scout' | 'sage' | 'maker' | 'butler';

export type AppPage =
  | 'ai-scout'
  | 'ai-sage'
  | 'ai-maker'
  | 'ai-butler'
  | 'new-conversation'
  | 'skill-library'
  | 'mcp-config'
  | 'automation'
  | 'knowledge-vault'
  | 'app-center'
  | 'usage-stats'
  | 'industry-report'
  | 'ai-policy'
  | 'industry-data';

export const AI_ROLE_PAGES: AppPage[] = ['ai-scout', 'ai-sage', 'ai-maker', 'ai-butler'];

export const ROLE_NAMES: Record<AIRole, string> = {
  scout: '探路者AI',
  sage: '军师AI',
  maker: '工匠AI',
  butler: '管家AI',
};

export const ROLE_ICONS: Record<AIRole, string> = {
  scout: 'SearchOutlined',
  sage: 'BulbOutlined',
  maker: 'ToolOutlined',
  butler: 'CustomerServiceOutlined',
};

export const PAGE_TO_ROLE: Record<string, AIRole> = {
  'ai-scout': 'scout',
  'ai-sage': 'sage',
  'ai-maker': 'maker',
  'ai-butler': 'butler',
};

export const ROLE_TO_PAGE: Record<AIRole, AppPage> = {
  scout: 'ai-scout',
  sage: 'ai-sage',
  maker: 'ai-maker',
  butler: 'ai-butler',
};

// ============ Skill 类型 ============
```

```
export type SkillCategory =
  | 'marketing'
  | 'analysis'
  | 'writing'
  | 'coding'
  | 'knowledge'
  | 'office'
  | 'design'
  | 'finance'
  | 'product'
  | 'automation'
  | 'custom';

export interface AutoTrigger {
  id: string;
  type: 'keyword' | 'intent' | 'regex';
  condition: string;
}

export interface Skill {
  id: string;
  name: string;
  description: string;
  category: SkillCategory;
  tags: string[];
  promptTemplate: string;
  triggerCommand: string;
  autoTriggers: AutoTrigger[];
  isEnabled: boolean;
  usageCount: number;
  createdAt: number;
  updatedAt: number;
}

// ============ MCP 类型 ============

export type MCPTransport = 'stdio' | 'sse';
export type MCPStatus = 'connected' | 'disconnected' | 'error' | 'connecting';

export interface MCPTool {
  name: string;
  description: string;
  inputSchema: unknown;
}

export interface MCPServer {
  id: string;
  name: string;
  transport: MCPTransport;
```

```
  command?: string;
  args?: string[];
  env?: Record<string, string>;
  url?: string;
  status: MCPStatus;
  tools: MCPTool[];
  configJson: string;
  createdAt: number;
  updatedAt: number;
}

// ============ 自动化类型 ============

export type TriggerType =
  | 'message-keyword'
  | 'conversation-start'
  | 'mcp-result'
  | 'schedule'
  | 'hook';

export type ActionType =
  | 'send-message'
  | 'invoke-skill'
  | 'invoke-mcp'
  | 'switch-role'
  | 'set-variable';

export interface AutomationAction {
  id: string;
  type: ActionType;
  config: Record<string, unknown>;
}

export interface TriggerConfig {
  type: TriggerType;
  config: Record<string, unknown>;
}

export interface AutomationRule {
  id: string;
  name: string;
  description: string;
  isEnabled: boolean;
  trigger: TriggerConfig;
  actions: AutomationAction[];
  maxIterations: number;
  runMode: 'sequential' | 'parallel';
  createdAt: number;
  updatedAt: number;
}
```

```
export interface ExecutionLog {
  id: string;
  ruleId: string;
  ruleName: string;
  status: 'success' | 'failed' | 'running';
  message: string;
  executedAt: number;
}

// ============ 用户与设置类型 ============

export type UserTier = 'free' | 'pro';

export interface UserInfo {
  id: string;
  nickname: string;
  avatar?: string;
  phone: string;
  tier: UserTier;
  tierLabel: string;
  quickPassCount: number;
}

export type ThemeType = 'light' | 'dark';
export type LanguageType = 'zh-CN' | 'en';

export interface AppSettings {
  theme: ThemeType;
  language: LanguageType;
  privacyMode: boolean;
  notificationsEnabled: boolean;
}

// ============ 模型配置类型 ============

export interface ModelConfig {
  id: string;
  name: string;
  provider: string;
  apiKey: string;
  modelId?: string;
  baseUrl?: string;
  contextWindowInput?: number;
  contextWindowOutput?: number;
  toolCallRounds?: number;
  multimodal: boolean;
  isCustom: boolean;
  isEnabled: boolean;
  createdAt: number;
```

```
  updatedAt: number;
}

export type SettingsTab = 'account' | 'general' | 'model';

// 预设模型列表（火山方舟 Agent Plan 套餐内模型）

export const PRESET_MODELS = [
  { name: 'doubao-seed-2.0-lite', provider: '火山方舟', modelId: 'doubao-seed-2.0-lite', baseUrl: 'https://ark.cn-beijing.vo
  { name: 'doubao-seed-2.0-mini', provider: '火山方舟', modelId: 'doubao-seed-2.0-mini', baseUrl: 'https://ark.cn-beijing.vo
  { name: 'doubao-seed-2.0-pro', provider: '火山方舟', modelId: 'doubao-seed-2.0-pro', baseUrl: 'https://ark.cn-beijing.volc
  { name: 'doubao-seed-2.0-code', provider: '火山方舟', modelId: 'doubao-seed-2.0-code', baseUrl: 'https://ark.cn-beijing.vo
  { name: 'deepseek-v4-pro', provider: '火山方舟', modelId: 'deepseek-v4-pro', baseUrl: 'https://ark.cn-beijing.volces.com/a
  { name: 'deepseek-v4-flash', provider: '火山方舟', modelId: 'deepseek-v4-flash', baseUrl: 'https://ark.cn-beijing.volces.c
  { name: 'kimi-k2.6', provider: '火山方舟', modelId: 'kimi-k2.6', baseUrl: 'https://ark.cn-beijing.volces.com/api/plan/v3',
  { name: 'kimi-k2.7-code', provider: '火山方舟', modelId: 'kimi-k2.7-code', baseUrl: 'https://ark.cn-beijing.volces.com/api
  { name: 'glm-5.2', provider: '火山方舟', modelId: 'glm-5.2', baseUrl: 'https://ark.cn-beijing.volces.com/api/plan/v3', mul
  { name: 'minimax-m2.7', provider: '火山方舟', modelId: 'minimax-m2.7', baseUrl: 'https://ark.cn-beijing.volces.com/api/pla
  { name: 'minimax-m3', provider: '火山方舟', modelId: 'minimax-m3', baseUrl: 'https://ark.cn-beijing.volces.com/api/plan/v3
];
```